ML-KEM-768 Private Key Exfiltration Verification Protocol

Document Version: 2.0

Date: October 24, 2026

Applies to: the Inheritor iOS release after 2.5.0

Purpose: Enable users to verify non-exfiltration of ML-KEM-768 private keys through network traffic analysis, and list every network endpoint the app contacts with what it sends

Draft status. The Helios consensus endpoints (Section 2.6) and the checkpoint services (Section 2.7) are **placeholders**: the public defaults the code ships with today, pending confirmation. This document is final only when those entries are confirmed or replaced.

Confidentiality: Public (for transparency)

Next Review: Upon any change to the key-provider request or the providers in Section 2, or when the placeholder entries are confirmed

Contents

TL;DR **3**

1 Threat Model **3**
1.1 Legitimate Behavior (Expected) . . . 3
1.2 Malicious Behavior (Exfiltration Attack) . . . 4

2 Network Endpoint Inventory **4**
2.1 Inheritor key provider (Cloudflare Worker) . . . 4
2.2 Infura (Ethereum and Arbitrum JSON-RPC) . . . 5
2.3 Pimlico (ERC-4337 bundler and paymaster) . . . 5
2.4 Arweave . . . 5
2.5 Verified claim check: independent pinned RPC providers . . . 6
2.6 Helios light client, Ethereum only . . . 6
2.7 Checkpoint services and network time (PLACEHOLDER) . . . 7
2.8 CoinGecko . . . 7
2.9 Apple services . . . 7
2.10 Not app traffic . . . 7

3 What Leaves the Device **8**

4 Size Analysis: Legitimate Key Provider Request **8**
4.1 JSON Payload Breakdown . . . 8
4.2 What is `encryptedSymmetricKey`? . . . 9

5 Size Analysis: ML-KEM-768 Private Key **9**

6 Measuring the Request **10**
6.1 What to measure . . . 10
6.2 TLS and HTTP overhead . . . 10

7 Detection Thresholds **11**
7.1 Safe Zones (request body to the key provider) . . . 11
7.2 Limitations . . . 11

8 User Verification Protocol **11**
8.1 Overview . . . 11
8.2 Option 1: Charles Proxy or Proxyman (Recommended for iOS) . . . 12
8.3 Option 2: Wireshark (Advanced Users, connection-level only) . . . 12
8.4 What to Look For . . . 12
8.4.1 Green Flags (Normal Behavior) . . . 12

8.4.2 Red Flags (Potential Exfiltration) 13
8.5 Checking the other endpoints 13

9 Technical Implementation Details **13**
9.1 URLSession Configuration 13
9.2 Cipher Suite Specifications 13
9.3 Published Commitment 14

Conclusion **14**

TL;DR

This document describes a simple and reliable protocol for users to verify that the Inheritor iOS application does **not** exfiltrate ML-KEM-768 private keys to the Inheritor key provider (a Cloudflare Worker) or any other server. The verification relies on a fundamental size difference:

- **Legitimate key provider request body:** 783–867 bytes (one 60-byte wrapped key plus metadata and two signatures)
- **ML-KEM-768 private key alone:** 2,400 bytes raw, 3,200 characters in base64
- **Request body carrying the key:** at least 3,900 bytes

Key Finding: Any request body to the key provider larger than 1,500 bytes is clear evidence that something other than the documented request is being sent, such as the private key.

Limitation: a small request does not prove that no key left the device. A deliberately compromised build could send a 64-byte key seed instead of the expanded key, or split a key over several requests or other endpoints (Section 7.2). Size monitoring catches an oversized key upload; it is not a proof of non-exfiltration.

Section 2 lists every endpoint the app contacts. As built from its source, apart from that one wrapped key, the only secrets that leave the device are the keys Apple's iCloud Keychain synchronises, end-to-end encrypted, between the user's own devices (Section 3).

1 Threat Model

1.1 Legitimate Behavior (Expected)

After an inheritance is created on-chain, the app sends **one** request to the key provider (`POST /v2/keys`, request version 2) to store the beneficiary's ML-KEM-768 wrapped key:

```
{
  "requestVersion": 2,
  "chainId": 42161,
  "deploymentIdentity": "0x…",       // 66 characters
  "inheritanceId": "0x…",            // 66 characters
  "beneficiary": "0x…",              // 42 characters
  "payloadFingerprint": "0x…",       // 66 characters
  "encryptedSymmetricKey": "…",      // 80 characters (base64)
  "timestamp": 1793000000,           // Unix seconds
  "signature": "0x…",                // 132 characters
  "appSignature": "0x…"              // 132 characters
}
```

Total JSON body: 783 bytes (Ethereum) or 787 bytes (Arbitrum), plus 1 byte for each `/` in the base64 value, which the iOS JSON encoder escapes as `\/` (usually 1–3; at most 80).

What this contains:

- `encryptedSymmetricKey`: the **60-byte wrappedK** of the ML-KEM-768 recipient (12-byte nonce + 32-byte wrapped AES key + 16-byte authentication tag), base64 encoded to 80 characters. This is the only secret in any request.
- Routing and binding data: chain, deployment, inheritance ID, beneficiary address and the fingerprint of the encrypted payload on Arweave.

- Two EIP-712 signatures over the same digest: one by the testator's Ethereum account (the testator's address can be recovered from it) and one by the app signer key, which is the same in every copy of the app and only serves as an anti-spam gate.

What this does NOT contain:

- ML-KEM-768 private key (2,400 bytes)
- ML-KEM-768 public key (1,184 bytes)
- X-Wing private key
- The BIP39 recovery phrase or any Ethereum private key
- The AES content key in unwrapped form, or any asset data

1.2 Malicious Behavior (Exfiltration Attack)

A compromised application could attempt to exfiltrate the ML-KEM-768 private key by:

1. **Replacing legitimate payload:** Substitute the 80-character `encryptedSymmetricKey` field with the 2,400-byte private key
2. **Adding extra field:** Include the private key as an additional JSON field
3. **Encoding in existing field:** Hide the private key in another field

Sending the expanded key in any of these ways produces a request body of at least 3,900 bytes, because the 2,400-byte key needs 3,200 characters in JSON. Sending a key seed instead, or splitting the key, does not (Section 7.2).

2 Network Endpoint Inventory

Every endpoint is a compile-time constant. Users cannot change any of them, and no setting, environment variable or Info.plist key overrides them. The app has **no analytics, crash reporting, advertising or tracking SDK**, sets no custom User-Agent (iOS sends its default `Inheritor/<build> CFNetwork/… Darwin/…`), and every provider below sees the device's IP address.

2.1 Inheritor key provider (Cloudflare Worker)

Host	`keyprovider-prod.inheritor.workers.dev` (App Store builds); `keyprovider-test.inheritor.workers.dev` (debug and TestFlight builds)
Requests	`POST /v2/keys`, header `Content-Type: application/json`
Sent	The 10-field body in Section 1: chain ID, deployment identity, inheritance ID, beneficiary address, payload fingerprint, the 60-byte ML-KEM wrappedK, Unix timestamp, testator signature, app signature
When	Once after the inheritance transaction is mined; again at a later launch only while the key is still pending (up to 4 attempts 3 s apart on 404/503, and after the wait the provider sets on 429)
Response used	Status code, and on failure `retryAfter`, `message` and an error code
Identifying	Inheritance ID, beneficiary address, testator address (from the signature), IP address

Worker's own chain reads	The worker checks each request against the contract through two providers of its own. In production these are Infura and Alchemy. They see the worker's contract reads about that inheritance (its ID and addresses), sent from Cloudflare's network, not the device's IP address.
IP address at the worker	The worker's own code neither stores nor logs the client IP address. It uses the address only as the name of a rate-limit bucket, which Cloudflare hashes into a Durable Object ID, and its log lines carry event names and paths. Cloudflare's Workers observability is enabled for every request (`head_sampling_rate = 1.0`), and Cloudflare's own request logs may include the client IP address.

The worker also has a `POST /v2/release` route (same fields minus the wrapped key and app signature, signed by the beneficiary). The app never calls it; external recovery tools do.

2.2 Infura (Ethereum and Arbitrum JSON-RPC)

Hosts	`mainnet.infura.io`, `arbitrum-mainnet.infura.io` (`/v3/<project ID>`; the project ID is the same in every copy of the app). `arbitrum-mainnet.infura.io` is also one of the pinned Arbitrum claim-check providers (2.5).
Methods	`eth_call`: registry, the testator's inheritance list (`digitalWill(testator, i)`, one read per index), the beneficiary's list (`getBeneficiaryInheritances(beneficiary)`), inheritance records (`inheritances(id)`, also at past block numbers to find an inheritance's creation date), smart-account links, verifier and testator state, nonces, EntryPoint `getNonce`. Also `eth_getCode`, `eth_gasPrice`, and `eth_getBlockByNumber` (block timestamps). No event-log queries.
When	Launch (registry, smart-account checks), sync, check-in, notification scheduling, lists of inheritances to verify or claim
Identifying	Testator and beneficiary addresses, smart-account addresses, inheritance IDs, all tied to the IP address. All of these are public on-chain.

2.3 Pimlico (ERC-4337 bundler and paymaster)

Hosts	`api.pimlico.io/v2/1/rpc`, `api.pimlico.io/v2/42161/rpc` (API key in the query string; the same in every copy of the app)
Methods	`pm_sponsorUserOperation`, `eth_estimateUserOperationGas`, `eth_sendUserOperation`, `eth_getUserOperationReceipt`, `pimlico_getUserOperationStatus`, `pimlico_getUserOperationGasPrice`
Sent	The user operation: smart account, nonce, init code (factory and owner address), gas fields, paymaster data, the owner's signature, and the contract call. For a new inheritance the call carries the inheritance ID, beneficiary addresses, the encrypted testator data (name and message sealed to the beneficiary), the Arweave transaction ID, the payload fingerprint, the check-in interval and grace period, and the app signature.
When	Every contract action: account creation, linking, check-in, adding an inheritance, claiming, verifier changes
Identifying	Everything sent becomes public on-chain once the operation is mined.

2.4 Arweave

Upload node	`arweave.net`: `GET /tx_anchor`, `GET /price/{bytes}`, `POST /tx`, `POST /chunk`
Read gateways	`turbo-gateway.com`, `ar-io.dev`, `arweave.net` (in that order): `GET /{id}` or `/raw/{id}`, `GET /tx/{id}/status`, `POST /graphql`
Sent (upload)	The encrypted asset envelope: ciphertext, nonce, tag, the file extension in plain text, and the recipients' KEM ciphertexts, salts and X-Wing wrapped keys (the ML-KEM wrapped key is left empty; it goes only to the key provider). The transaction is signed by the app's own Arweave wallet, shared by every copy of the app, with tags `App-Name`, `App-Version` and `Content-Type`. The upload size reveals the approximate file size.
When	Upload: creating an inheritance. Reads: viewing or claiming an inheritance, and confirmation polling.
Every launch	A read-health check: a GraphQL query for the 3 latest `App-Name: Inheritor` transactions, then a download of one of them (at most 1 MB). This fetches someone else's public, encrypted payload; it sends nothing about the user.

2.5 Verified claim check: independent pinned RPC providers

Ethereum	Keyed: Alchemy (`eth-mainnet.g.alchemy.com`), dRPC (`lb.drpc.live`). Keyless spares: `ethereum-rpc.publicnode.com`, `mainnet.gateway.tenderly.co`, `eth.drpc.org`, `eth.rpc.blxrbdn.com`
Arbitrum	Keyed: Alchemy (`arb-mainnet.g.alchemy.com`), Infura (`arbitrum-mainnet.infura.io`, also the general RPC in 2.2). Keyless spares: `arb1.arbitrum.io/rpc`, `arbitrum.gateway.tenderly.co`, `arbitrum-one.public.blastapi.io`, `arb-pokt.nodies.app`
Keys	The keyed providers' API keys are in the URL path and are the same in every copy of the app
Selection	The first two providers run by different operators that answer, in the order above; the results must agree
Transport	TLS pinned to the root keys each host chains to (Let's Encrypt and Google Trust Services; Sectigo for keyed dRPC; Amazon for Infura); ephemeral session (no cache, no cookies), redirects refused
Methods	`eth_chainId`, `eth_getBlockByNumber` (`finalized`, `latest`), `eth_call` pinned by block hash with `requireCanonical`: registry `getContractAddress`, `inheritances(id)`, `contractPaused()`, `getVersion()`
When	Claiming, the contract version check, and decrypting testator metadata. On Ethereum only when the Helios light client (2.6) cannot give a trusted answer.
Identifying	Inheritance IDs, IP address

2.6 Helios light client, Ethereum only

Placeholder: the consensus endpoints below are the public defaults, pending confirmation. The execution providers are final.

Execution providers	Keyed: Alchemy (`eth-mainnet.g.alchemy.com`), dRPC (`lb.drpc.live`). Keyless spares: `eth.drpc.org`, `eth.rpc.blxrbdn.com`
Methods	Preflight `eth_chainId`, `eth_getBlockByNumber("finalized")`, `eth_getProof` for the zero address; then `eth_getProof` pinned by block hash, `eth_getCode`, `eth_createAccessList` (sends the contract address and call data, so the inheritance ID), `eth_getBlockByNumber` / `eth_getBlockByHash`

Consensus endpoints (placeholder)	`ethereum.operationsolarstorm.org`, `lodestar-mainnet.chainsafe.io`: `GET /eth/v1/beacon/light_client/bootstrap/{root}`, `/updates`, `/finality_update`, `/optimistic_update`, `/eth/v1/config/spec`. No user data.
Transport	System trust store and DNS resolver, no pinning; every response is verified against Ethereum's sync-committee signatures and Merkle proofs
When	Each time the app comes to the foreground (light-client sync), and for claim and metadata reads
Identifying	Inheritance IDs (execution providers only), IP address

2.7 Checkpoint services and network time (PLACEHOLDER)

Placeholder: the six services below are the current defaults, pending confirmation.

Hosts	`sync-mainnet.beaconcha.in`, `mainnet.checkpoint.sigp.io`, `beaconstate.ethstaker.cc`, `mainnet-checkpoint-sync.attestant.io`, `mainnet-checkpoint-sync.stakely.io`, `beaconstate-mainnet.chainsafe.io`
Request	`GET /eth/v1/beacon/states/finalized/finality_checkpoints`, header `Accept: application/json`. No body.
Transport	TLS pinned (Let's Encrypt; Stakely: Google Trust Services); ephemeral session
Use	A checkpoint at least 3 operators agree on, fetched only when the stored checkpoint is stale; and trusted time from their HTTP `Date` headers, refreshed every 10 minutes and on each sync. The phone's clock is never used for freshness.
Identifying	IP address only

2.8 CoinGecko

`GET api.coingecko.com/api/v3/simple/price?ids=ethereum&vs_currencies=usd` for gas-cost estimates in the credits view. No user data; the result is cached.

2.9 Apple services

- **App Attest** (generating a beneficiary code): iOS sends Apple the attestation key and a hash of a random challenge. No beneficiary data. The resulting attestation goes into the code the user shares themselves. Verifying an imported code makes no network request.
- **StoreKit** (subscriptions and credits): handled by iOS with Apple. Inheritor runs no purchase server.
- **iCloud Keychain**: see Section 3.

2.10 Not app traffic

- **Links the user opens:** quick-start guide, premium and resilience pages, the Telegram support channel, the whitepaper, the privacy policy and terms, and block explorers (Etherscan, Arbiscan, ViewBlock). They open in the browser.
- **Local sharing:** MultipeerConnectivity between nearby devices (encryption required) carries only the beneficiary code. The peer name is `Inheritor-` plus 4 characters of the vendor identifier. Nothing goes through a server.

3 What Leaves the Device

Secrets. Two kinds of secret leave the device:

1. The 60-byte ML-KEM wrappedK, to the key provider (Section 2.1). It is useless without the beneficiary's ML-KEM private key.
2. The BIP39 recovery phrase, the X-Wing and ML-KEM private keys and the verified beneficiary records. They are stored in the keychain with iCloud synchronisation, so Apple's iCloud Keychain carries them, end-to-end encrypted, to the user's own devices. Inheritor's servers never receive them.

The App Attest identity stays on the device. No other private key, seed, content key or plaintext asset is sent anywhere.

Public or encrypted data. Encrypted assets go to Arweave; the encrypted testator data and the inheritance parameters go on-chain through the bundler; both are public and permanent by design.

Identifying data. Ethereum and smart-account addresses, inheritance IDs and the IP address reach the providers in Section 2. Addresses and inheritance IDs are public on-chain anyway. The testator's and beneficiary's names, messages and files never leave the device unencrypted.

4 Size Analysis: Legitimate Key Provider Request

4.1 JSON Payload Breakdown

From `KeyProviderRequest.swift` (`StoreKey.body`):

```
try JSONSerialization.data(withJSONObject: [
    "requestVersion": KeyProviderRequest.requestVersion,         // 2
    "chainId": chainId,                                           // 1 or 42161
    "deploymentIdentity": KeyProviderRequest.hex(deploymentIdentity),
    "inheritanceId": KeyProviderRequest.hex(inheritanceId),
    "beneficiary": beneficiary,
    "payloadFingerprint": KeyProviderRequest.hex(payloadFingerprint),
    "encryptedSymmetricKey": encryptedSymmetricKey.base64EncodedString(),
    "timestamp": timestamp,
    "signature": KeyProviderRequest.hex(signatures.signature),
    "appSignature": KeyProviderRequest.hex(signatures.appSignature)
] as [String: Any])
```

Field	Size (bytes, with quotes)	Notes
`requestVersion`	1	2
`chainId`	1 or 5	1 (Ethereum) or 42161 (Arbitrum)
`deploymentIdentity`	68	bytes32: 0x + 64 hex
`inheritanceId`	68	bytes32: 0x + 64 hex
`beneficiary`	44	address: 0x + 40 hex
`payloadFingerprint`	68	bytes32: 0x + 64 hex
`encryptedSymmetricKey`	82	60 bytes (wrappedK) as 80 base64 characters
`timestamp`	10	Unix seconds
`signature`	134	65-byte ECDSA: 0x + 130 hex

Field	Size (bytes, with quotes)	Notes
`appSignature`	134	65-byte ECDSA: `0x` + 130 hex

Field values: 610 bytes (Ethereum) or 614 bytes (Arbitrum)

Field names with quotes and colons: 162 bytes. **Braces and commas:** 11 bytes.

Total JSON body: 610 + 162 + 11 = **783 bytes** (Ethereum), **787 bytes** (Arbitrum). Each `/` in the base64 value adds 1 byte, so the absolute maximum is 867 bytes.

4.2 What is `encryptedSymmetricKey`?

From `QSCryptionManager.swift`, the key-provider data is the **ML-KEM-768 recipient's wrappedK field**:

```
struct RecipientInfo: Codable {
    let recV: Int          // Recipient format version
    let type: String       // "mlkem768"
    let kid: String        // Key identifier
    let kem_ct: Data       // KEM encapsulation (1088 bytes)
    let salt: Data         // HKDF salt (32 bytes)
    let wrappedK: Data     // THIS is what goes to the key provider (60 bytes)
}
```

wrappedK structure (60 bytes):

- 12 bytes: AES-GCM nonce
- 32 bytes: AES-256 symmetric key (wrapped/encrypted)
- 16 bytes: AES-GCM authentication tag

Base64 encoding: 60 bytes $\rightarrow$ 80 characters

This is a wrapped AES key that protects the actual asset data. The ML-KEM recipient entry in the Arweave envelope keeps an empty wrappedK, so the key provider holds the only copy.

5 Size Analysis: ML-KEM-768 Private Key

From `CryptionConfiguration.swift`:

```
/// ML-KEM-768 secret key size (for external tool compatibility)
public static let mlKem768SecretKeySize = 2400
```

ML-KEM-768 key sizes (NIST FIPS 203 standard):

- Private key (decapsulation key): **2,400 bytes** (fixed)
- Public key (encapsulation key): **1,184 bytes** (fixed)
- Ciphertext (KEM encapsulation): **1,088 bytes** (fixed)
- Shared secret: **32 bytes** (fixed)

If exfiltrated as base64: 2,400 bytes → 3,200 characters

Size comparison:

- Legitimate `encryptedSymmetricKey`: 80 characters (60 bytes raw)
- Exfiltrated ML-KEM private key: 3,200 characters (2,400 bytes raw)
- **Ratio:** 40x larger

A body with the key in place of the wrappedK is **3,903–3,907 bytes**; with the key added as an extra field it is larger still.

6 Measuring the Request

6.1 What to measure

Measure the **size of the request body** (the `Content-Length`, or the request body size a proxy shows). Do not compare single network packets: a TCP segment rarely exceeds the 1,500-byte Ethernet MTU, so a large request is split over several packets and no single packet is ever "too big".

6.2 TLS and HTTP overhead

When the body is sent over HTTPS, TLS 1.3 adds a 5-byte record header and a 16-byte authentication tag per record, and the HTTP request line and headers add about 150–250 bytes (HTTP/1.1; less with HTTP/2 header compression):

```
POST /v2/keys HTTP/1.1
Host: keyprovider-prod.inheritor.workers.dev
Content-Type: application/json
Content-Length: 787
Accept: */*
Accept-Language: en-GB,en;q=0.9
Accept-Encoding: gzip, deflate, br
User-Agent: Inheritor/<build> CFNetwork/<version> Darwin/<version>
```

```
Legitimate request (application data in the TLS stream):
┌──────────────────────────────────┐
│ HTTP request line + headers: ~250 │
├──────────────────────────────────┤
│ JSON body: 783–867 bytes          │
├──────────────────────────────────┤
│ TLS record headers + tags: ~21–42 │
└──────────────────────────────────┘
Total: ~1,050–1,160 bytes

Exfiltration request:
┌──────────────────────────────────┐
│ HTTP request line + headers: ~250 │
├──────────────────────────────────┤
│ JSON body: ≥ 3,903 bytes          │
├──────────────────────────────────┤
│ TLS record headers + tags: ~21–42 │
└──────────────────────────────────┘
Total: ≥ 4,170 bytes
```

7 Detection Thresholds

7.1 Safe Zones (request body to the key provider)

Zone	Body size	Interpretation
Safe	≤ 900 bytes	Legitimate request
Suspicious	900–1,500 bytes	Investigate (unexpected fields)
Exfiltration	> 1,500 bytes	Clear evidence of private key exfiltration

Rationale:

- Legitimate bodies: at most 867 bytes
- Bodies with the ML-KEM key: at least 3,903 bytes
- Threshold: 1,500 bytes, with a margin of more than 600 bytes on the legitimate side and 2,400 bytes on the other

False positive rate: Zero for request version 2 (the body cannot exceed 867 bytes)

Detection: a request carrying the expanded 2,400-byte key cannot stay under the threshold. For a single request the same separation holds in the TLS stream (Section 6): about 1,160 bytes at most against at least 4,170. A connection that carries several requests (retries) adds up, so only a per-request measurement (Option 1 in Section 8) supports a verdict.

7.2 Limitations

Request sizes detect only an attack that sends the expanded private key in one request. They cannot rule out:

1. **A key seed.** ML-KEM-768 key generation (FIPS 203) derives the whole key pair from two 32-byte random values. A compromised build could send those 64 bytes, or only the 32-byte seed d, which is enough to decrypt. That fits inside the 80-character `encryptedSymmetricKey` field, which is random-looking by design, so neither size nor content would show it.
2. **Fragmentation.** A key split over many requests, or over several inheritances, keeps every request under the threshold.
3. **Other channels.** Any other endpoint in Section 2 could carry key material: for example inside an Arweave upload or a bundler request (both public, so they can be inspected after the fact), or in request headers.

The size check is therefore a cheap test against a careless or naive exfiltration, not a proof. Stronger assurance needs the app's source code and a build that can be compared with it.

8 User Verification Protocol

8.1 Overview

Users can verify non-exfiltration by capturing network traffic while creating an inheritance and checking the size of the key-provider request.

Tools required:

- A network capture tool (Charles Proxy, Proxyman, or Wireshark)

- Basic ability to filter and inspect requests

Time required: 5–10 minutes per verification

Technical skill level: Intermediate (can use packet capture tools)

8.2 Option 1: Charles Proxy or Proxyman (Recommended for iOS)

1. **Install the proxy** on macOS and configure the iOS device to use it (Settings → Wi-Fi → Configure Proxy).
2. **Enable SSL proxying** for `keyprovider-prod.inheritor.workers.dev` and install the proxy's root certificate on the device.
    - Do **not** enable SSL proxying for the claim-check providers (2.5) or the checkpoint services (2.7). Their TLS connections are pinned, so they refuse the proxy's certificate and the app reports the claim check as unavailable. That refusal is the pinning working as intended. This includes `arbitrum-mainnet.infura.io`: proxying it breaks the Arbitrum claim check, while the app's other Infura calls keep working.
3. **Start recording** and open Inheritor.
4. **Create a test inheritance** with a small test asset (e.g. a 1 KB text file) and complete the process.
5. **Inspect the key-provider request:** filter for `keyprovider-prod.inheritor.workers.dev`, select the `POST /v2/keys` request, and read the request body size.
6. **Verify the size:**
    - **Expected:** 783–867 bytes (legitimate)
    - **Red flag:** > 1,500 bytes (exfiltration)

8.3 Option 2: Wireshark (Advanced Users, connection-level only)

Without TLS decryption Wireshark sees connections, not individual requests, so it cannot give the per-request verdict of Option 1. Use it to see which hosts the app talks to and roughly how much it sends.

1. **Install Wireshark** on macOS.
2. **Capture iOS traffic** through a remote virtual interface: `rvictl -s <device-UDID>`, then capture on `rvi0`.
3. **Find the key-provider connection:** `tls.handshake.extensions_server_name contains "keyprovider"` identifies it.
4. **Read the bytes sent** on that connection (Statistics → Conversations shows bytes per direction).
5. **Interpret with care:** one store request sends about 1,050–1,200 bytes of application data. The app makes up to 4 attempts at a store (3 s apart, on 404 or 503) and URLSession may reuse the connection, so a connection can legitimately carry several requests. A larger total is a reason to repeat the check with Option 1, not evidence of exfiltration by itself.

8.4 What to Look For

8.4.1 Green Flags (Normal Behavior)

- Exactly one `POST /v2/keys` per new inheritance, with a body under 900 bytes

- No requests to hosts not listed in Section 2
- Consistent sizes across several inheritance creations

8.4.2 Red Flags (Potential Exfiltration)

- Any key-provider request body over 1,500 bytes
- Several large requests to the key provider for one inheritance
- Requests to hosts not listed in Section 2
- A `POST /v2/release` from the app (the app never sends one)

8.5 Checking the other endpoints

- **Arweave uploads** are large by nature (they carry the encrypted file). They are public: download the transaction (`https://arweave.net/raw/<id>`) and check that it is the documented envelope, with the ML-KEM wrappedK empty.
- **Bundler requests** carry the contract call, which is published on-chain once mined; decode the transaction on Etherscan or Arbiscan to see exactly what was sent.
- **JSON-RPC, light-client and checkpoint requests** are small (well under 2 KB) and carry addresses, inheritance IDs and block identifiers only.

9 Technical Implementation Details

9.1 URLSession Configuration

- The key provider, Arweave, CoinGecko and some Infura calls use `URLSession.shared`; web3swift's Infura calls and the bundler use `URLSessionConfiguration.default`. These use iOS's default TLS settings (TLS 1.2 minimum, TLS 1.3 preferred) and the system trust store.
- The claim-check providers and the checkpoint services use a pinned, ephemeral session: TLS 1.2 or later, the server's certificate chain must contain one of the pinned root keys, no cache, no cookies, no redirects.
- The Helios light client (Rust) uses the system trust store and the system DNS resolver; it trusts no response it cannot verify cryptographically.

iOS defaults:

- TLS 1.3 preferred (with TLS 1.2 fallback)
- AEAD cipher suites (AES-GCM, ChaCha20-Poly1305)
- Perfect Forward Secrecy (PFS) enabled

9.2 Cipher Suite Specifications

TLS 1.3 cipher suites (iOS 13+):

1. `TLS_AES_256_GCM_SHA384` or `TLS_AES_128_GCM_SHA256`
    - Symmetric: AES-GCM
    - Auth tag: 16 bytes
    - Key exchange: X25519 or secp256r1 (or the hybrid X25519MLKEM768 on recent iOS versions)

2. `TLS_CHACHA20_POLY1305_SHA256`
    - Symmetric: ChaCha20-Poly1305
    - Auth tag: 16 bytes

Overhead consistency: All of these use 16-byte authentication tags, so the overhead is predictable.

9.3 Published Commitment

Inheritor (key provider request version 2) commits to:

```
Key Provider Request Size Guarantee (POST /v2/keys):
═══════════════════════════════════════
JSON body:            783–787 bytes (+1 per escaped "/")
MAXIMUM BODY:         867 bytes
SAFE UPPER BOUND:     900 bytes
═══════════════════════════════════════

Exfiltration Detection:
Any key-provider request body > 1,500 bytes = private key exfiltration
A body carrying the ML-KEM-768 key is at least 3,903 bytes
```

The size bound follows from the fixed field lengths of request version 2. It shows that the documented request cannot carry the expanded ML-KEM-768 key; it does not exclude the cases in Section 7.2.

Conclusion

The Inheritor app's exfiltration verification protocol detects an upload of the expanded ML-KEM-768 private key to the key provider through simple request size monitoring:

Key Findings:

1. **Legitimate key-provider request body:** 783–867 bytes
2. **Body carrying the key:** at least 3,903 bytes
3. **Detection ratio:** at least 4.5x
4. **Detection threshold:** 1,500 bytes
5. **False positive rate:** zero for request version 2
6. **Limits:** a 32- or 64-byte key seed, a key split over several requests, or another channel would not be caught (Section 7.2)

Endpoint transparency: Section 2 lists every host the app contacts and what it sends. Anything else in a capture is a red flag.

User Verification:

Users with basic capture skills can run this check in 5–10 minutes with free tools. It catches an oversized key upload even with limited networking knowledge, but it is one check among several, not a proof that no key material leaves the device.